HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Office for International Study Programs

Introduction to System on Chip
CO3143

Lab Report

Week 1: Introduction & Arithmetic

Lecturer: Phạm Kiều Nhật Anh
Class: CC02
Student: Nguyễn Viết Tấn Vương - 2353345

Ho Chi Minh City, 2025

Contents

1 Exercise 1: Traffic Light System **2**
1.1 Introduction and System Design 2
1.2 Source Code Implementation 2
1.3 Waveform Analysis 5

2 Exercise 2: LED Decoration System **6**
2.1 Introduction and System Design 6
2.2 Source Code Implementation 6
2.3 Waveform Analysis 8

3 Exercise 3: LED String System with UART **8**
3.1 Introduction and System Design 8
3.2 Source Code Implementation 8
3.3 Waveform Analysis 12

4 Homework: 32-bit Unsigned Divider **12**
4.1 Algorithm Introduction 12
4.2 Source Code Implementation 12
4.3 Waveform Analysis 13

1 Exercise 1: Traffic Light System

1.1 Introduction and System Design

This exercise involves designing a traffic light control system for an intersection. The system simulates two traffic light poles using RGB LEDs (Red, Green, Yellow) and displays a countdown timer on 7-segment LEDs.
The system operates in two modes controlled by a switch (`sw[0]`):

- **Normal Mode (sw[0] = 0):** The traffic lights operate automatically, cycling through Green, Yellow, and Red states based on a countdown timer.
- **Configuration Mode (sw[0] = 1):** The user can adjust the duration of the Green and Yellow lights using buttons.

1.2 Source Code Implementation

The module `traffic_light_system` implements the control logic. It uses a Finite State Machine (FSM) with four states: `S_G1_R2`, `S_Y1_R2`, `S_R1_G2`, and `S_R1_Y2`.

```
`timescale 1ns / 1ps

module traffic_light_system #(
    parameter CLOCK_LIMIT = 62500000
)(
    input  wire       clk,          // System Clock (125MHz)
    input  wire       reset_n,      // Reset Input (Active High logic based on original code)
    input  wire [1:0] sw,           // SW0: Mode (0: Run, 1: Config), SW1: Config Select
    input  wire [1:0] btn,          // BTN1: Time Increment Button

    // Output: RGB LEDs (Traffic Light Colors)
    output reg  [2:0] led_rgb1,
    output reg  [2:0] led_rgb2,

    // Output: 7-Segment Display (BCD)
    output wire [3:0] seg0_bcd,     // Units Digit
    output wire [3:0] seg1_bcd,     // Tens Digit
    output wire [3:0] seg2_bcd,     // Unused (OFF)
    output wire [3:0] seg3_bcd      // Unused (OFF)
);

    // --- INTERNAL SIGNALS ---
    wire rst;
    assign rst = reset_n; // Map internal reset to input

    // Finite State Machine (FSM) States
    localparam S_G1_R2 = 2'b00; // Lane 1 Green,  Lane 2 Red
    localparam S_Y1_R2 = 2'b01; // Lane 1 Yellow, Lane 2 Red
    localparam S_R1_G2 = 2'b10; // Lane 1 Red,    Lane 2 Green
    localparam S_R1_Y2 = 2'b11; // Lane 1 Red,    Lane 2 Yellow

    reg [1:0] current_state;
    reg [5:0] timer_count;      // Countdown timer (6-bit)

    // Configuration Registers (Default: Green=10s, Yellow=3s)
```

```
    reg [5:0] cfg_green_time;
    reg [5:0] cfg_yellow_time;

    // Clock Divider Signals
    reg [26:0] clk_counter;
    reg        clk_1hz;

    // Button Edge Detection
    reg  btn1_prev;
    wire btn1_posedge;


    always @(posedge clk) begin
        btn1_prev <= btn[1];
    end
    assign btn1_posedge = btn[1] && !btn1_prev;


    always @(posedge clk or posedge rst) begin
        if (rst) begin
            cfg_green_time  <= 10;
            cfg_yellow_time <= 3;
        end else begin
            // Only active in Config Mode (SW[0] = 1)
            if (sw[0] == 1) begin
                if (btn1_posedge) begin
                    if (sw[1] == 0) begin
                        // SW1=0: Adjust Green Light (Wrap 60 -> 5)
                        if (cfg_green_time < 60)
                            cfg_green_time <= cfg_green_time + 1;
                        else
                            cfg_green_time <= 5;
                    end else begin
                        // SW1=1: Adjust Yellow Light (Wrap 20 -> 2)
                        if (cfg_yellow_time < 20)
                            cfg_yellow_time <= cfg_yellow_time + 1;
                        else
                            cfg_yellow_time <= 2;
                    end
                end
            end
        end
    end


    always @(posedge clk or posedge rst) begin
        if (rst) begin
            clk_counter <= 0;
            clk_1hz     <= 0;
        end else begin
            if (clk_counter >= CLOCK_LIMIT - 1) begin
                clk_counter <= 0;
                clk_1hz     <= ~clk_1hz;
            end else begin
                clk_counter <= clk_counter + 1;
            end
        end
    end

```

```

    always @(posedge clk_1hz or posedge rst) begin
        if (rst) begin
            current_state <= S_G1_R2;
            timer_count   <= 9; // Initial timer value
        end else begin
            // FSM only runs in Run Mode (SW[0] = 0)
            if (sw[0] == 0) begin
                if (timer_count == 0) begin
                    // State Transition Logic
                    case (current_state)
                        S_G1_R2: begin
                            current_state <= S_Y1_R2;
                            timer_count   <= cfg_yellow_time - 1;
                        end
                        S_Y1_R2: begin
                            current_state <= S_R1_G2;
                            timer_count   <= cfg_green_time - 1;
                        end
                        S_R1_G2: begin
                            current_state <= S_R1_Y2;
                            timer_count   <= cfg_yellow_time - 1;
                        end
                        S_R1_Y2: begin
                            current_state <= S_G1_R2;
                            timer_count   <= cfg_green_time - 1;
                        end
                    endcase
                end else begin
                    // Decrement timer
                    timer_count <= timer_count - 1;
                end
            end
        end
    end


    always @(*) begin
        // If in Config Mode: White Light Indicator
        if (sw[0] == 1) begin
             led_rgb1 = 3'b111; // White
             led_rgb2 = 3'b111; // White
        end else begin
            // Run Mode: FSM controlled colors
            // Encoding: 010=Green, 001=Yellow, 100=Red
            case (current_state)
                S_G1_R2: begin led_rgb1 = 3'b010; led_rgb2 = 3'b100; end // Green - Red
                S_Y1_R2: begin led_rgb1 = 3'b001; led_rgb2 = 3'b100; end // Yellow - Red
                S_R1_G2: begin led_rgb1 = 3'b100; led_rgb2 = 3'b010; end // Red - Green
                S_R1_Y2: begin led_rgb1 = 3'b100; led_rgb2 = 3'b001; end // Red - Yellow
                default: begin led_rgb1 = 3'b000; led_rgb2 = 3'b000; end
            endcase
        end
    end

```

```
    reg [5:0] display_value;

    // Display Mux
    always @(*) begin
        if (sw[0] == 1) begin
            // Config Mode: Display the setting value
            if (sw[1] == 0) display_value = cfg_green_time;
            else            display_value = cfg_yellow_time;
        end else begin
            // Run Mode: Display countdown (+1 for human readability)
            display_value = timer_count + 1;
        end
    end

    // Binary to BCD Conversion
    wire [3:0] bcd_tens = display_value / 10;
    wire [3:0] bcd_units = display_value % 10;

    // Output Assignments
    assign seg1_bcd = bcd_tens;  // Tens Digit
    assign seg0_bcd = bcd_units; // Units Digit

    assign seg2_bcd = 4'd0;      // OFF
    assign seg3_bcd = 4'd0;      // OFF

endmodule
```

Listing 1: Traffic Light System Module (Ex_1.v)

1.3 Waveform Analysis

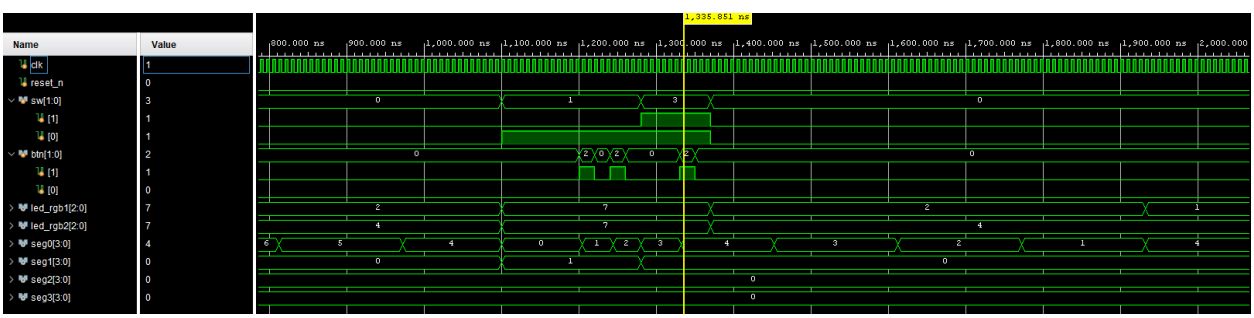

Figure 1: Simulation Waveform for Exercise 1

The simulation results in Figure 1 demonstrate the system's operation:

1. **Configuration:** Initially, `sw[0]` is high. The user presses buttons to increment the setup time.

2. **Operation:** When `sw[0]` goes low, the system enters normal operation.

3. **State Transitions:** The RGB LEDs (`led_rgb1`, `led_rgb2`) change values sequentially. For example, `led_rgb1` transitions from Green (2) to Yellow (1) and then to Red (4), while `led_rgb2` holds Red (4) before switching to Green.

4. **Countdown:** The 7-segment output (`seg0`, `seg1`) displays the countdown timer decrementing each second.

2 Exercise 2: LED Decoration System

2.1 Introduction and System Design

This exercise implements a decoration system using 7-segment LEDs to create visual effects. The system displays two numbers (2 and 5) moving across the four 7-segment displays.
The movement is controlled by switches (`sw`):

- `sw=00`: Shift Left.
- `sw=01`: Shift Right.
- `sw=10`: Bounce Mode (automatically reverses direction at the edges).

2.2 Source Code Implementation

The `led_decoration_system` module uses a counter to generate a movement tick and a state machine to update the position (`step`) of the numbers.

```
`timescale 1ns / 1ps

module led_decoration_system #(parameter CLOCK_LIMIT = 25000000) (
    input clk,
    input reset_n,
    input [1:0] sw,

    // Output 4 7-segment LEDs
    output reg [3:0] seg0_bcd,
    output reg [3:0] seg1_bcd,
    output reg [3:0] seg2_bcd,
    output reg [3:0] seg3_bcd
);

    wire rst = reset_n;

    localparam NUM_3 = 4'd3;
    localparam NUM_6 = 4'd6;
    localparam BLANK = 4'hF;

    reg [25:0] counter;
    reg move_tick;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            counter <= 0;
            move_tick <= 0;
        end else begin
            if (counter >= CLOCK_LIMIT - 1) begin
                counter <= 0;
                move_tick <= 1;
            end else begin
                counter <= counter + 1;
                move_tick <= 0;
```

```
            end
        end
    end

    reg [1:0] step;
    reg direction;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            step <= 0;
            direction <= 0;
        end else if (move_tick) begin

            case (sw)
                2'b00: begin
                    step <= step - 1;
                end

                2'b01: begin
                    step <= step + 1;
                end

                2'b10: begin
                    if (direction == 0) begin
                        if (step == 2) begin
                            direction <= 1;
                            step <= 1;
                        end else begin
                            step <= step + 1;
                        end
                    end else begin
                        if (step == 0) begin
                            direction <= 0;
                            step <= 1;
                        end else begin
                            step <= step - 1;
                        end
                    end
                end

                default: step <= 0;
            endcase
        end
    end

    always @(*) begin
        seg3_bcd = BLANK; seg2_bcd = BLANK; seg1_bcd = BLANK; seg0_bcd = BLANK;

        case (step)
            0: begin
                seg3_bcd = NUM_3; seg2_bcd = NUM_6;
            end
            1: begin
                seg2_bcd = NUM_3; seg1_bcd = NUM_6;
            end
            2: begin
                seg1_bcd = NUM_3; seg0_bcd = NUM_6;
            end
            3: begin
```

```
                seg0_bcd = NUM_3; seg3_bcd = NUM_6;
            end
        endcase
    end

endmodule
```

Listing 2: LED Decoration Module (ex_2.v)

2.3 Waveform Analysis

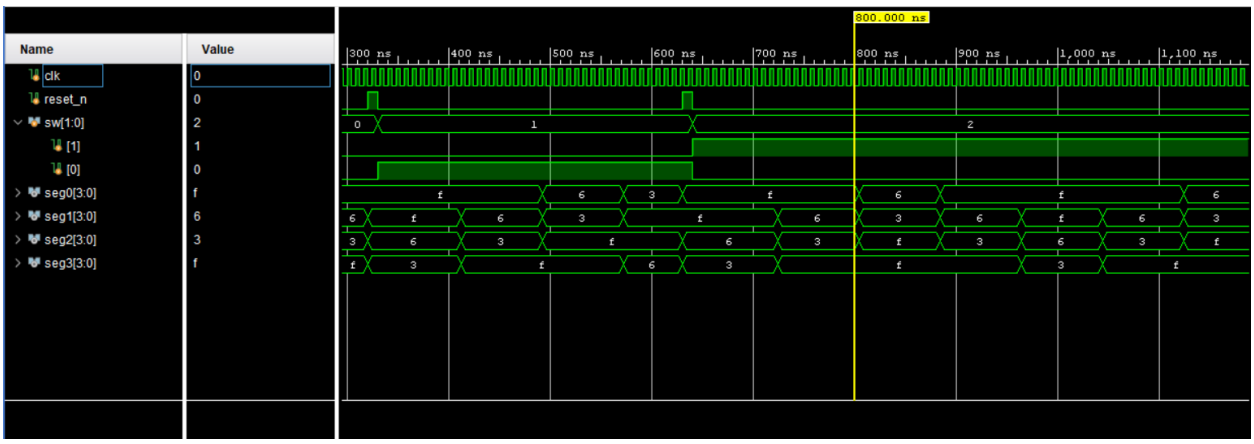

Figure 2: Simulation Waveform for Exercise 2

The waveform in Figure 2 shows the three modes of operation:

1. **Shift Left:** Initially, with `sw=00`, the segments change such that the numbers appear to move left.
2. **Shift Right:** When `sw` changes to `01`, the pattern reverses.
3. **Bounce:** Finally, with `sw=10`, the `step` variable oscillates between 0 and 3, creating a back-and-forth motion without manual intervention.

3 Exercise 3: LED String System with UART

3.1 Introduction and System Design

This exercise combines LED string control with UART communication. The system controls a 4-bit LED string that can shift left, shift right, or pause based on button inputs. Additionally, the current state of the LEDs is transmitted to a PC via UART every second.

3.2 Source Code Implementation

The system consists of the main module `led_string_system` and a UART transmitter module `uart_tx`. The main module handles the LED shifting logic and instantiates the UART transmitter to send the LED status string (e.g., "1100" followed by a Line Feed).

```
module led_string_system #(
    parameter CLOCK_FREQ = 125000000,
    parameter BAUD_RATE  = 9600
)(
    input wire clk,
    input wire reset_n,
    input wire [3:1] btn,
    output reg [3:0] led,
    output wire uart_tx_out
);

    localparam CNT_MAX_VAL = CLOCK_FREQ - 1;
    localparam ASCII_ZERO  = 8'h30;
    localparam ASCII_ONE   = 8'h31;
    localparam ASCII_LF    = 8'h0A;

    reg [1:0] sys_mode;
    reg [31:0] timer_cnt;
    reg pulse_1s;

    reg tx_req;
    reg [7:0] tx_data;
    wire tx_busy;

    reg [3:0] fsm_state;
    reg [3:0] led_buffer;

    always @(posedge clk or posedge reset_n) begin
        if (reset_n) begin
            sys_mode <= 2'b00;
        end else begin
            if (btn[3])
                sys_mode <= 2'b00;
            else if (btn[1])
                sys_mode <= 2'b01;
            else if (btn[2])
                sys_mode <= 2'b10;
        end
    end

    always @(posedge clk or posedge reset_n) begin
        if (reset_n) begin
            timer_cnt <= 32'd0;
            pulse_1s <= 1'b0;
        end else begin
            if (timer_cnt >= CNT_MAX_VAL) begin
                timer_cnt <= 32'd0;
                pulse_1s <= 1'b1;
            end else begin
                timer_cnt <= timer_cnt + 1;
                pulse_1s <= 1'b0;
            end
        end
    end

    always @(posedge clk or posedge reset_n) begin
        if (reset_n) begin
            led <= 4'b0011;
        end else if (pulse_1s) begin
```

```
            case (sys_mode)
                2'b01: led <= {led[2:0], led[3]};
                2'b10: led <= {led[0], led[3:1]};
                default: led <= led;
            endcase
        end
    end

    uart_tx #(
        .CLOCK_FREQ(CLOCK_FREQ),
        .BAUD_RATE(BAUD_RATE)
    ) inst_uart_tx (
        .clk(clk),
        .rst(reset_n),
        .start(tx_req),
        .data(tx_data),
        .tx(uart_tx_out),
        .busy(tx_busy)
    );

    always @(posedge clk or posedge reset_n) begin
        if (reset_n) begin
            fsm_state <= 4'd0;
            tx_req <= 1'b0;
            tx_data <= 8'd0;
        end else begin
            case (fsm_state)
                4'd0: begin
                    tx_req <= 1'b0;
                    if (pulse_1s) begin
                        led_buffer <= led;
                        fsm_state <= 4'd1;
                    end
                end

                4'd1: begin
                    if (!tx_busy) begin
                        tx_data <= (led_buffer[3]) ? ASCII_ONE : ASCII_ZERO;
                        tx_req <= 1'b1;
                        fsm_state <= 4'd2;
                    end
                end

                4'd2: begin
                    tx_req <= 1'b0;
                    fsm_state <= 4'd3;
                end

                4'd3: begin
                    if (!tx_busy) begin
                        tx_data <= (led_buffer[2]) ? ASCII_ONE : ASCII_ZERO;
                        tx_req <= 1'b1;
                        fsm_state <= 4'd4;
                    end
                end

                4'd4: begin
                    tx_req <= 1'b0;
                    fsm_state <= 4'd5;
```

```
                end

                4'd5: begin
                    if (!tx_busy) begin
                        tx_data <= (led_buffer[1]) ? ASCII_ONE : ASCII_ZERO;
                        tx_req <= 1'b1;
                        fsm_state <= 4'd6;
                    end
                end

                4'd6: begin
                    tx_req <= 1'b0;
                    fsm_state <= 4'd7;
                end

                4'd7: begin
                    if (!tx_busy) begin
                        tx_data <= (led_buffer[0]) ? ASCII_ONE : ASCII_ZERO;
                        tx_req <= 1'b1;
                        fsm_state <= 4'd8;
                    end
                end

                4'd8: begin
                    tx_req <= 1'b0;
                    fsm_state <= 4'd9;
                end

                4'd9: begin
                    if (!tx_busy) begin
                        tx_data <= ASCII_LF;
                        tx_req <= 1'b1;
                        fsm_state <= 4'd10;
                    end
                end

                4'd10: begin
                    tx_req <= 1'b0;
                    fsm_state <= 4'd11;
                end

                4'd11: begin
                    if (!tx_busy)
                        fsm_state <= 4'd0;
                end

                default: fsm_state <= 4'd0;
            endcase
        end
    end

endmodule
```

Listing 3: LED String System (ex_3.v)

3.3 Waveform Analysis

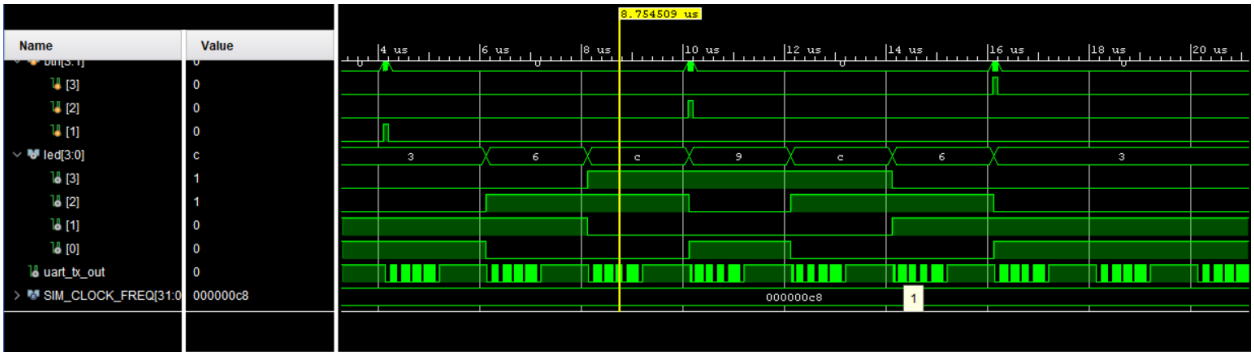

Figure 3: Simulation Waveform for Exercise 3

Figure 3 illustrates the interaction between user input and system output:

1. **Mode Selection:** Pulses on `btn[1]` and `btn[2]` change the internal mode.
2. **LED Update:** The `led` signal updates every second (triggered by `tick_1hz`).
3. **UART Output:** The `uart_tx_out` signal shows activity immediately after every LED update. This represents the serial data stream sending the character representation of the LEDs to the computer.

4 Homework: 32-bit Unsigned Divider

4.1 Algorithm Introduction

The homework objective is to design a 32-bit unsigned divider for the RISC-V processor. Since Verilog division operators are restricted, a combinational "shift-and-subtract" algorithm is implemented. The design unrolls the iterative division loop into 32 cascaded stages to perform the calculation in a single cycle.

4.2 Source Code Implementation

The design is modular, consisting of a single-iteration module (`divu_1iter`) instantiated 32 times in the top-level module (`divider_unsigned`).

```
module divu_1iter(
    input [31:0] i_dividend, i_divisor, i_remainder, i_quotient,
    output reg [31:0] o_dividend, o_divisor, o_remainder, o_quotient
);
    always @(*) begin
        // Shift Remainder left, shift in MSB of Dividend
        remainder_shifted = (i_remainder << 1) | ((i_dividend >> 31) & 1'b1);
        o_dividend = i_dividend << 1;

        // Compare and Subtract
        if (remainder_shifted < i_divisor) begin
            o_quotient = i_quotient << 1;
            o_remainder = remainder_shifted;
```

```
        end else begin
            o_quotient = (i_quotient << 1) | 1'b1;
            o_remainder = remainder_shifted - i_divisor;
        end
    end
endmodule
```

Listing 4: Single Iteration Module (divu_1iter.v)

```
module divider_unsigned (
    input  wire [31:0] i_dividend, i_divisor,
    output wire [31:0] o_remainder, o_quotient
);
    wire [31:0] dividend_chain [32:0];
    wire [31:0] remainder_chain [32:0];
    wire [31:0] quotient_chain [32:0];

    genvar i;
    generate
        for (i = 0; i < 32; i = i + 1) begin : div_loop
            divu_1iter inst (
                .i_dividend (dividend_chain[i]),
                .i_divisor  (i_divisor),
                // ... connecting chains
                .o_quotient (quotient_chain[i+1])
            );
        end
    endgenerate

    assign o_quotient  = quotient_chain[32];
    assign o_remainder = remainder_chain[32];
endmodule
```

Listing 5: Unsigned Divider Module (divider_unsigned.v)

4.3 Waveform Analysis

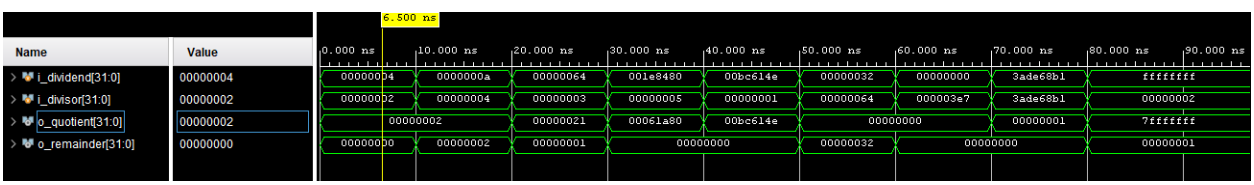

Figure 4: Simulation Waveform for Homework (Unsigned Divider)

The simulation in Figure 4 verifies the combinational divider logic across various test cases:

1. **Basic Division:** At 0ns, 4/2 yields Quotient=2, Remainder=0.
2. **Remainder Check:** At 10ns, 10/4 yields Quotient=2, Remainder=2.
3. **Large Numbers:** At 30ns, $2,000,000/5$ yields $400,000$.
4. **Edge Case:** At 50ns, 50/100 correctly yields Quotient=0 and Remainder=50.

These results confirm that the 32-stage combinational logic correctly implements 32-bit integer division.

— End —